Architectural Tradeoffs in Low-Latency Algorithmic Trading with Predictive Signals

Abhinav Randive

February 18, 2026

Modern financial markets are dominated by firms that employ algorithmic trading systems that operate under strict real-time constraints. In particular, high-frequency and low-latency trading systems rely heavily on sophisticated software architectures that integrate market data processing, predictive modelling, and order execution within microsecond. While predictive models for financial markets have been widely studied, existing research emphasizes that prediction accuracy alone is insufficient; execution latency, system design, and market microstructure effects more often than not determine whether a predictive edge can be realized in practice [1, 5]. This project proposes the design and implementation of a low-latency algorithmic trading system that integrates short-term predictive models with an event-driven execution engine, allowing for a quantitative study of the trade-offs between prediction accuracy, latency, and execution quality.

Prior work in algorithmic trading highlights the importance of market microstructure, including limit order book dynamics, queue position, and adverse selection [3]. At the same time, research in high-performance systems demonstrates that architectural decisions such as concurrency models, memory layout, and event handling strategies can significantly impact tail latency and throughput [4]. Despite this, many academic trading studies abstract away execution mechanics or rely on unrealistic assumptions about fills and transaction costs. This project addresses that gap by focusing explicitly on the software infrastructure required to deploy prediction-aware trading strategies under realistic execution constraints.

The primary goal of this project is to build a modular, event-driven trading system capable of replaying historical market data, generating real-time predictions from order book features, and executing trades within a simulated limit order book. The system will not be evaluated primarily on profitability, but rather on engineering-centric metrics such as latency distributions, order fill rates, slippage, and sensitivity to execution delays. By doing so, the project aims to demonstrate how even accurate predictive signals can lose effectiveness when system latency or execution costs are too high, a phenomenon well documented in both academic and industry research [2].

The software will be structured as a multi-component system. At its core will be a limit order book implementation supporting limit orders, market orders, and cancellations using price–time priority. An event-driven engine will process timestamped market data events and route them through the system deterministically, ensuring reproducible experiments and eliminating look-ahead bias. A feature extraction module will compute microstructure-based features such as order book imbalance, spread, and short-term order flow statistics.

These features will feed into lightweight predictive models, such as logistic regression or linear classifiers, designed to predict short-horizon price movements or volatility changes. Predictions will then inform a strategy layer that controls order placement, aggressiveness, and cancellation behavior.

The execution simulator will model realistic trading constraints, including queue position, partial fills, and transaction costs. Instrumentation will be built into every stage of the pipeline to measure end-to-end latency, component-level delays, and tail latency behavior under load. This instrumentation enables systematic experimentation on how changes in model complexity, feature computation cost, or architectural design affect overall system performance.

This project is worth accomplishing because it bridges the gap between predictive modeling and systems engineering in algorithmic trading. By treating execution and latency as first-class research concerns, the project aligns closely with real-world financial engineering practice while remaining firmly grounded in computer science. The resulting system will serve both as a research platform for controlled experimentation and as a demonstration of advanced software engineering skills in performance-critical environments. When implemented, the system's high-level logic flow will illustrate how market data enters the system, how predictions are generated in real time, how trading decisions are made, and how orders interact with the simulated market. This flow will be represented as a logic diagram (Figure 1) showing the interaction between the data feed, prediction module, strategy logic, and execution engine.

Appendix

A planned feature list for the software system is outlined below. Features are listed in the intended order of implementation.

- Historical market data ingestion and deterministic replay engine.
- Event-driven processing loop with timestamped events.
- Core limit order book implementation with price–time priority.
- Support for limit orders, market orders, and order cancellations.
- Market microstructure feature extraction (spread, depth, imbalance).
- Baseline predictive model for short-horizon price direction.
- Strategy module that adjusts order placement based on predictions.
- Execution simulator modeling queue position and partial fills.
- Transaction cost and slippage modeling.
- End-to-end latency measurement and profiling infrastructure.
- Visualization of order book state and latency distributions.
- Parameterized experiments comparing prediction accuracy vs latency.
- Stretch goal: parallelized or lock-free event queue implementation.
- Stretch goal: comparison of multiple prediction models under latency constraints.

References

[1] Aldridge, I. *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*, 2nd ed. John Wiley & Sons, 2013.

[2] Bouchaud, J.-P., Farmer, J. D., and Lillo, F. How markets slowly digest changes in supply and demand. *Handbook of Financial Markets: Dynamics and Evolution* (2009), 57–160.

[3] Harris, L. *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, 2003.

[4] Stonebraker, M., Çetintemel, U., and Zdonik, S. The 8 requirements of real-time stream processing. *Association for Computing Machinery Special Interest Group on Management of Data Record 34*, 4 (2005), 42–47.

[5] Álvaro Cartea, Jaimungal, S., and Penalva, J. *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015.

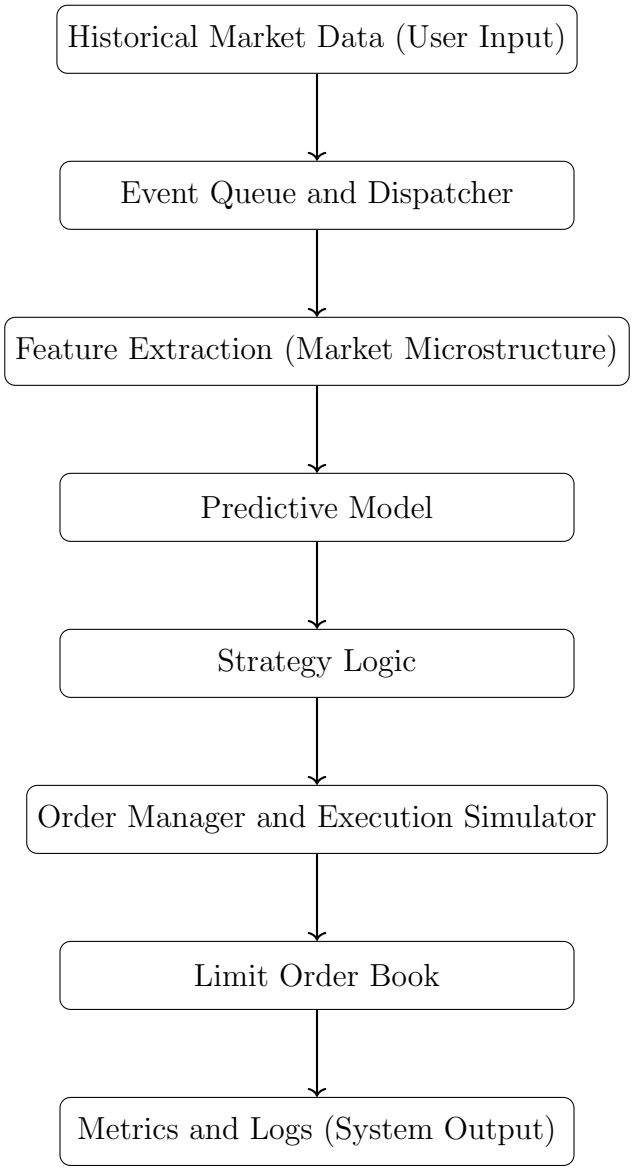

Figure 1: Event-driven Architecture Model

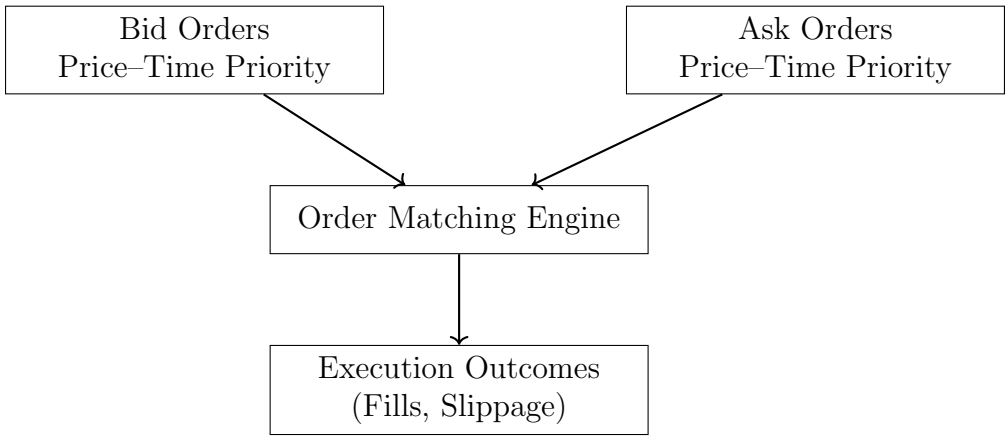

Figure 2: Structure of the simulated limit order book